

KNOW-A-GUY

Milestone 1 Technical Architecture Report

Team: Michael Gubler

Date: September 26, 2026

Chapter 1 | Overview

Problem

Life is full of many not-so-welcome surprises. Household appliances randomly seem to stop working, basements flood under heavy rain storms, plumbing starts to struggle, and many other types of issues occur where you wish you knew a guy that could come over and fix whatever is broken. You could go through private companies or word of mouth to potentially find someone who is able to resolve your dilemma. But then you start to think, “Is this the best price I can get? Is there someone out there who can do the same or better job for less cost?”

Solution | Know-a-Guy

Know-a-Guy is a web based platform that aggregates publicly accessible local professionals on popular sites such as TaskRabbit and/or Thumbtack and even larger agencies to find the best man for the job at the best price. Not only that, but independent contractors will be able to make their own accounts directly in the platform to try to stand out on their own and set their own competitive rates.

Product experience

- Describe the problem in plain language, optionally attach photos, and provide location and timing requirements.
- Search approved external sources and Know-a-Guy contractor profiles using normalized service category, geography, price signals, ratings, and availability when available.
- Return a comparable result set with source attribution so customers can distinguish direct profiles from aggregated listings.
- Let independent contractors create direct profiles so supply is not limited to large marketplaces.

Architecture | Use Cases and Data Flow

User stories

- As a homeowner, I want to describe a household problem and optionally upload photos so that Know-a-Guy can return relevant local professionals with comparable price and profile information.
- As an independent contractor, I want to create a profile with services, service area, rate guidance, and contact details so that I can compete directly for relevant jobs.
- As a platform operator, I want connector workers to refresh and normalize source listings on a schedule so that comparisons use traceable, recently observed data.

End-to-end workflow

1. Request intake: validate job data; upload photos to object storage using an API-issued upload target.
2. Orchestration: create the job request, identify candidate sources, and enqueue source lookups.
3. Aggregation: source connectors fetch permitted data and map it into a common listing schema with source + observed_at provenance.
4. Matching: score candidates using job type, geography, price signal, reputation signal, and availability when present.
5. Retrieval: persist normalized results and return a comparison view; cache hot reads where useful.

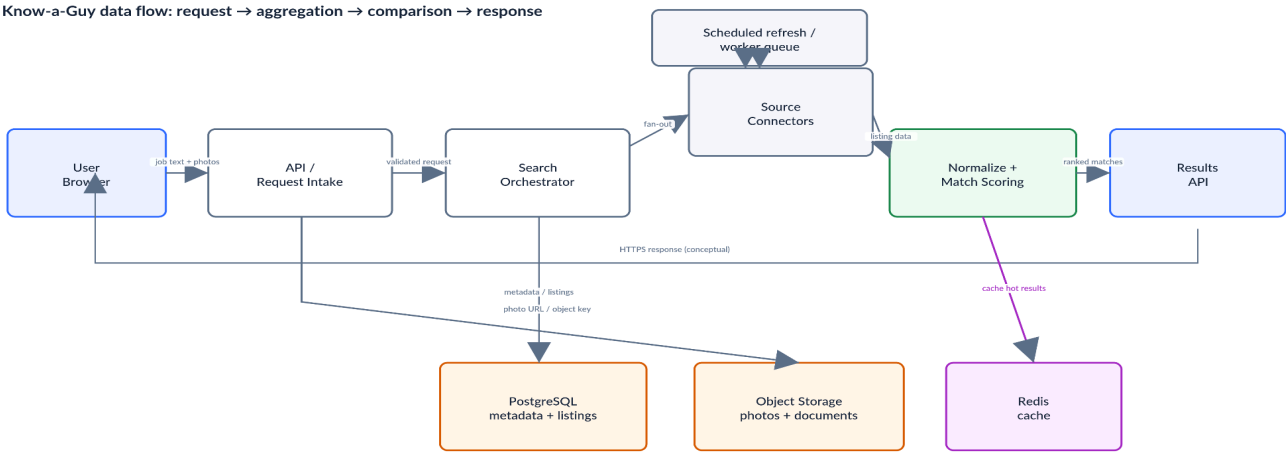


Figure 1. Data-flow diagram for a customer request and the source-refresh path.

Architecture | Kubernetes Design

Kubernetes architecture: stateless services scale independently; state is durable and isolated

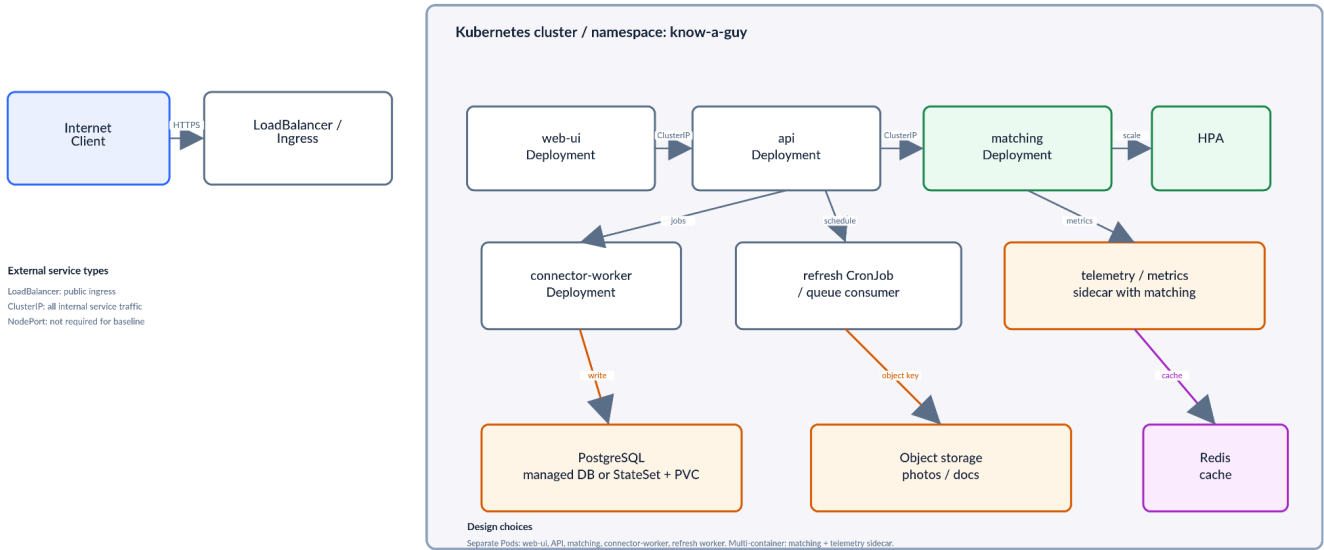


Figure 2. Proposed Kubernetes deployment boundaries, service exposure, and persistence relationships.

Pod and workload decisions

Workload	Kind	Pod model	Design choice
web-ui	Deployment	1 container	Stateless frontend; horizontal scaling is straightforward
api	Deployment	1 app container	Stateless HTTP boundary for validation and orchestration
matching	Deployment	app + metrics/telemetry sidecar	First scaling target; sidecar isolates telemetry concerns
connector-worker	Deployment	1 worker container	Provider-specific jobs scale independently
refresh-scheduler	CronJob	1 container/run	Periodic refresh without a permanent scheduler service
PostgreSQL	Managed DB	DB policy	Durable relational state
Redis	Managed cache or Deployment	1 container	Optional TTL cache; never source of truth

The web frontend, API gateway/backend, and preprocessing service are deployed as separate Pods because they have different responsibilities, resource requirements, and scaling needs. The matching service is also isolated in its own Pod and is designed to scale horizontally with Horizontal Pod Autoscaler (HPA) as the number of users and active requests increases, since matching and ranking are expected to be one of the more resource-intensive stages. The matching Pod may use a multi-container design with a metrics sidecar so application performance can be monitored without adding monitoring logic directly to the model container. Kubernetes Services provide stable networking between components: internal services use ClusterIP, while the application entry point uses a LoadBalancer to expose the platform externally. An Ingress controller can route HTTPS traffic to the appropriate service and provide a single public endpoint. Stateful data such as user accounts, professional profiles, job requests, and search

results are kept outside ephemeral Pods in persistent storage, using PostgreSQL for structured data and object storage for uploaded images or other large files. This architecture allows individual services to be updated or scaled independently, improves fault isolation, and provides a foundation for future features such as automated matching, additional data-source connectors, and increased traffic.

Architecture | API Contracts

Public API surface

Method	Endpoint	Purpose	Success
POST	/v1/job-requests	Create service request; start matching	202
GET	/v1/job-requests/{id}	Read request status + summary	200
GET	/v1/job-requests/{id}/matches	Read ranked candidates + provenance	200
POST	/v1/contractors	Create first-party contractor profile	201
PATCH	/v1/contractors/{id}	Update profile/services/rates	200
GET	/v1/contractors?service=&zip=	Search first-party contractors	200

Example payloads

POST /v1/job-requests

```
{
  "service": "plumbing",
  "description": "Basement drain backs up during heavy rain",
  "location": {"postal_code": "19403"},
  "urgency": "today",
  "photo_keys": ["jobs/8a2f/photo-1.jpg"]
}
```

GET /v1/job-requests/{id}/matches - response body

```
{
  "job_id": "jr_8a2f",
  "generated_at": "2026-09-26T18:00:00Z",
  "matches": [{
    "contractor_id": "ct_103",
    "source": "direct",
    "estimated_price": {"min": 120, "max": 180, "currency": "USD"},
  ]
}
```

```
"distance_miles": 4.7,  
"freshness_minutes": 18  
}  
}
```

Architecture | Persistence, Risk, and Test Plan

Persistence plan

Data class	System of record	Notes
Users / auth profile	PostgreSQL	Minimize personal fields
Job requests / statuses	PostgreSQL	Lifecycle + normalized job data
Contractor profiles	PostgreSQL	First-party source of truth
Listings / observations	PostgreSQL	Source ID + normalized fields + observed_at
Match scores	PostgreSQL	Version formula inputs for reproducibility
Photos / documents	Object storage	DB stores object key, MIME type, checksum, size
Hot reads	Redis	TTL cache only
Connector scratch	emptyDir or short-lived PVC	Non-authoritative staging

Top risks and mitigations

Risk	Impact	Mitigation
Source access changes / outage	High	Connector abstraction; feature flags; partial-result behavior
Price is not directly comparable	High	Label exact vs estimated; store range, unit, source timestamp
Stale / duplicate listings	High	Deduplicate by source listing ID + identity keys; refresh timestamps
Rate limits / timeouts	High	Queue, backoff, per-source concurrency limits, circuit breaking
Unclear ranking behavior	Medium	Version scoring formula; keep major factors auditable

Database loss / corruption	High	Automated backup + restore drill + migration versioning
----------------------------	------	---

Initial test matrix

Scenario	Expected result
Happy path: valid request + image	202; matching starts; source + timestamp present
No matches	Successful empty state, not 500
Source timeout	Partial results; source marked degraded
Duplicate listing	Single normalized listing; observation timestamp updated
Invalid contractor signup	422 validation; no partial persistence
10x burst traffic	Graceful degradation; matching replicas/queue grow

Chapter 2 | Implementation Plan

The project will be implemented as a web application with a frontend, backend services, persistent storage, and external data connectors. The frontend will provide the primary user interface for submitting requests and viewing professional results. The backend will expose APIs that handle user requests, retrieve professional information, and perform the matching and ranking logic. PostgreSQL will be used to store structured application data such as users, professionals, service categories, job requests, and search results. Larger files, such as photographs submitted by users, will be stored separately using object-based storage.

The first version of the matching system will use straightforward criteria rather than a complex machine-learning model. A submitted job will be categorized using information provided by the user, and available professionals will be filtered and ranked using factors such as service category, location, price, ratings, and availability. This approach allows the team to produce a working recommendation system while leaving room for a more sophisticated recommendation or inference model in a later version.

External professional listings will be incorporated through connector services. These connectors will retrieve information from approved external sources and transform it into a common Know-a-Guy format. A background refresh process will periodically update this information so that listings do not become outdated. Independent contractors will also be able to create and maintain their own profiles directly through the application.

The implementation will be completed in stages, with each stage producing a usable portion of the system.

Timeline	Phase	Planned Work
Weeks 1–2	Foundation	Set up the repository, development environment, database schema, application containers, and Kubernetes

		configuration.
Weeks 3–4	Core Application	Implement user accounts, service-request submission, backend APIs, and the basic frontend workflow.
Weeks 5–6	Matching System	Implement professional profiles, search/filtering, and the initial matching and ranking logic.
Weeks 7–8	Data Aggregation	Implement the external-source connector and refresh process, including normalization of professional data.
Weeks 9–10	Contractor Profiles	Add independent-contractor registration, profile management, pricing, and inclusion in search results.
Weeks 11–12	Testing & Deployment	Perform integration and load testing, fix issues, validate Kubernetes behavior, improve monitoring, and prepare the final deployment and demonstration.